
Guião Laboratorial

Aula 05 — Servidor BLE em Arduino

Tecnologias de Comunicação para a Internet das Coisas

Ano letivo 2026–2027

Catarina Silva

ESTGA, Universidade de Aveiro

Duração prevista: 2 horas

Índice

Enquadramento	2
Objetivos de Aprendizagem	2
1 Montagem do Periférico	3
2 Verificação por Linha de Comandos	3
3 Notificações e Visualização	5
Questões para Discussão	6
Referências	6

Enquadramento

A Aula 04 estabeleceu os conceitos de GAP e GATT e percorreu, com ferramentas de linha de comandos, a hierarquia de serviços e características de um periférico. Esta sessão constrói o periférico: um Arduino equipado com o módulo HM-11 passa a expor as leituras de um sensor através de notificações BLE, e um cliente recolhe-as e encaminha-as para visualização.

Os dois programas necessários são fornecidos na pasta `codigo/` e destinam-se a ser carregados e executados, não escritos. O trabalho do estudante consiste na montagem, na verificação por linha de comandos e na construção do fluxo de visualização em Node-RED.

Material

- Arduino Uno com shield BLE HM-11 e sensor DHT11.
- Biblioteca Arduino `DHTlib`, a mesma instalada na Aula 02. O *sketch* fornecido não compila sem ela.
- Raspberry Pi com BlueZ e a biblioteca `bluepy` instalados.
- Node-RED no Raspberry Pi, com o pacote `@flowfuse/node-red-dashboard`.

Todo o equipamento é o já utilizado em aulas anteriores.

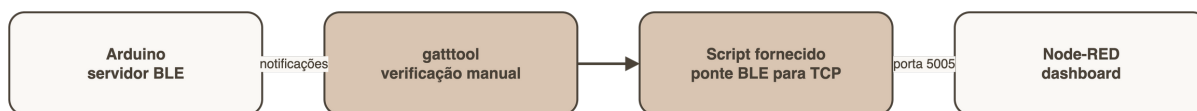


Figura 1: Arquitetura da cadeia completa

Objetivos de Aprendizagem

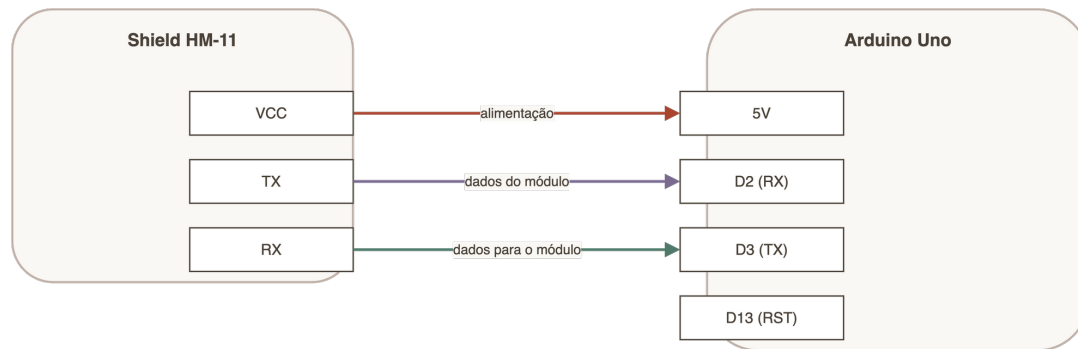
- Montar um periférico BLE e verificar que se anuncia corretamente;
- Confirmar por linha de comandos, antes de qualquer programa, que os serviços e características esperados estão expostos;
- Distinguir, na prática, a leitura por interrogação da receção por notificação;
- Integrar as leituras recebidas num painel de visualização construído sem escrita de código.

1. Montagem do Periférico

25 min

Objetivo

Montar o shield HM-11 e o sensor sobre o Arduino, e confirmar o funcionamento pelo monitor série antes de envolver a comunicação BLE.



O pino D13 comanda o reset do módulo no arranque

Figura 2: Ligações do shield HM-11 ao Arduino

Módulo	Arduino	Função
VCC	5V	Alimentação
TX	D2	Dados do módulo para o Arduino
RX	D3	Dados do Arduino para o módulo
RST	D13	Reinicialização no arranque
GND	GND	Massa
DHT11 DATA	D7	Leitura de temperatura e humidade

1. Efetuar as ligações com o Arduino desligado do cabo USB.
2. Carregar o ficheiro `codigo/servidor_ble.ino`, fornecido com este guião.
3. Abrir o monitor série a 9600 bit/s e enviar o carácter **X**. Deve ser devolvida uma linha com temperatura e humidade separadas por vírgula.

Porquê verificar primeiro pelo monitor série

Nesta fase a comunicação BLE ainda não está em causa. Se a leitura do sensor não funcionar aqui, qualquer falha observada mais tarde seria atribuída erradamente à ligação BLE. Esta ordem de verificação é o método de trabalho recomendado na componente teórica.

2. Verificação por Linha de Comandos

30 min

Objetivo

Confirmar, a partir do Raspberry Pi e sem recorrer a qualquer programa, que o periférico se anuncia e expõe as características esperadas.

```
hcitool dev
sudo hcitool lescan
sudo gatttool -b <ENDERECO> -I
```

No modo interativo do **gatttool**:

```
connect
primary
characteristics
char-write-req <handle> 58
```

1. Registrar o endereço do módulo HM-11 obtido no varrimento.
2. Registrar o UUID do serviço exposto pelo módulo e o *handle* da característica de dados.
3. O comando **char-write-req** envia o valor 0x58, que corresponde ao carácter X. Confirmar que o Arduino responde.

Leitura do resultado

Os valores devolvidos surgem em hexadecimal. A conversão para texto é imediata: cada par de dígitos corresponde a um carácter da resposta enviada pelo Arduino.

3. Notificações e Visualização

45 min

Objetivo

Executar a ponte fornecida, que ativa as notificações BLE e as disponibiliza numa porta TCP local, e construir em Node-RED o painel que as apresenta.

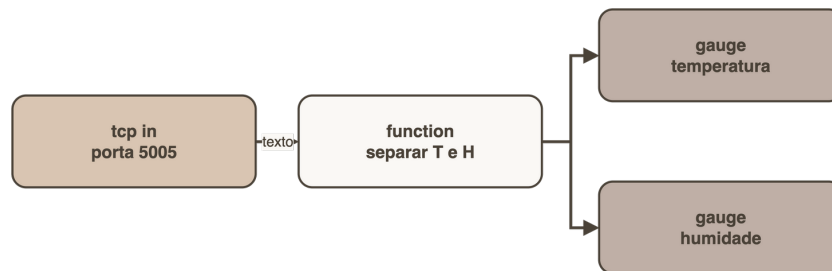


Figura 3: Fluxo Node-RED a construir

Execução da ponte

```
python3 codigo/ponte_ble_tcp.py <ENDERECO>
```

O programa liga-se ao periférico, ativa as notificações e escuta em 127.0.0.1:5005. As linhas recebidas são também apresentadas na consola, o que permite confirmar o funcionamento antes de envolver o Node-RED.

Construção do fluxo

1. Arrastar um nó `tcp in`. Configurar o modo *Connect to*, o anfitrião 127.0.0.1, a porta 5005 e a entrega de *strings* delimitadas por `\n`.
2. Arrastar um nó `function` com duas saídas e colar o conteúdo fornecido:

```
var v = msg.payload.split(",");
return [ { payload: parseFloat(v[0]) },
        { payload: parseFloat(v[1]) } ];
```

3. Associar a primeira saída a um `ui-gauge` designado *Temperatura* e a segunda a um `ui-gauge` designado *Humidade relativa*.
4. Premir `Deploy` e aceder a `http://127.0.0.1:1880/dashboard`.

Onde está a poupança de energia

O Arduino só transmite quando recebe um pedido, e a ponte recebe as respostas sob a forma de notificações, sem interrogar repetidamente a característica. Comparar este comportamento com o de uma leitura periódica permite observar a diferença discutida na componente teórica.

Trabalho complementar

Alterar o intervalo de pedidos na ponte fornecida, de um para dez segundos, e observar o efeito no painel. Que compromisso se estabelece entre atualidade dos dados e tráfego rádio?

Questões para Discussão

1. Por que razão foi necessária uma porta série por software no Arduino?
2. Qual a diferença entre notificação e indicação, e qual delas está a ser usada?
3. Que papel GATT desempenha o Arduino e qual desempenha o Raspberry Pi?
4. Se o Arduino deixasse de responder, ao fim de quanto tempo a ligação seria considerada perdida, e que parâmetro determina esse valor?

Referências

- Gonçalves, P. *Apresent5 — BLE Dev e guia 4 — arduinoBLE*, TCIoT, edições anteriores.
- Documentação da biblioteca bluepy: <https://ianharvey.github.io/bluepy-doc/>
- Nós TCP do Node-RED: <https://nodered.org/docs/user-guide/>